1. FULL PROJECT SRS (Complete SaaS-Ready Desktop ERP)

1.1 Purpose & Scope

Purpose: Build a **modular, desktop ERP** for scientific equipment importers, extensible to **multi-tenant SaaS**. Core modules: Inventory, Sales, Purchase, Import, Users, Branches.edrawmax+1

Scope (locked to prevent creep):

- Desktop app (Windows/macOS/Linux via Tauri).
- Offline-first, SQLite/PostgreSQL storage.
- Modular sidebar (company-enabled modules).
- RBAC (roles: Admin, Inventory Manager, Sales User, Import Clerk).
- **No:** Mobile app, real-time sync, payments gateway (v2).workos+1

Out of Scope: Web version, AI (phase 2), CRM, HR modules.

1.2 Users & Roles

Role	Permissions
Admin	All CRUD, module config, user/branch management.
Inventory Manager	Inventory/stock CRUD, view sales/reports.
Sales User	Sales/customers CRUD, view inventory.
Import Clerk	Import/purchase CRUD, view stock.

1.3 Functional Requirements (Modules)

- **User Management:** CRUD users, roles, permissions.
- **Branch Management:** CRUD branches/warehouses.
- **Inventory:** CRUD items (code, name, serial, price, category), stock per branch, low-stock alerts.gleek+1
- **Sales:** CRUD customers, invoices, invoice items (auto-stock deduct).
- **Purchase:** CRUD suppliers, purchase orders, goods receipt (stock add).
- **Import:** CRUD import orders (customs, shipping, duties), link to purchase.
- **Reports:** Dashboard stats, sales/inventory reports (PDF export).

- **Settings:** Company profile, module enable/disable.codewave+1

1.4 Non-Functional

- **UI:** Mantine + React (dark/light theme).
- **Backend:** Rust (Axum/SQLx).
- **Performance:** <2s page loads, handle 10k inventory items.
- **Security:** JWT auth, SQL injection prevention.
- **Deployment:** Tauri installer, auto-updates.

---

2. 2-WEEK PROTOTYPE SRS (University Demo + Client POC)

2.1 Purpose & Scope

Purpose: Working prototype with UI + basic CRUD for core flows, demo-ready for university/client. Focus: Inventory + Sales + Users.studocu+1

Scope (strict 2 weeks):

- Login + dashboard.
- **3 modules:** Users, Inventory, Sales.
- Basic CRUD only (no reports, imports yet).
- Hard-coded company/roles initially.
- SQLite only.

Deliverables by Week 2:

- Tauri desktop app (installer).
- Demo flows: Add user → Add inventory → Create sale (stock deducts).

Out of Scope: Branches, Purchase/Import, reports, multi-tenant.

2.2 Users & Roles (Simplified)

- Admin (full access).
- Sales User (sales only).

2.3 Functional Requirements (MVP)

- **Auth:** Login/logout (hard-coded users).
- **Users:** List/add/edit users (name, role).
- **Inventory:** List/add/edit items (name, code, price, qty), view stock.
- **Sales:** List/add invoice (customer name, items, total; deduct stock).
- **Dashboard:** Stats cards (total sales, low stock).kladana+1

2.4 Non-Functional (Prototype)

- Basic Mantine UI.
- Rust API via Tauri.
- No error handling polish needed.

---

3. ERDs (Entity Relationship Diagrams)

3.1 Full Project ERD (Textual + Mermaid)

Core entities + relationships. Use **Draw.io** to visualize.edrawmax+3

text

```
erDiagram
    companies ||--o{ users : "has"
    companies ||--o{ branches : "has"
    companies ||--o{ inventory_items : "has"
    companies ||--o{ customers : "has"
    roles ||--o{ users : "assigned_to"
    roles }o--o{ role_permissions : "has"
    branches ||--o{ stock : "has"
    inventory_items ||--o{ stock : "tracked_in"
    customers ||--o{ sales_invoices : "places"
    sales_invoices ||--o{ sales_items : "contains"
    inventory_items ||--o{ sales_items : "sold_as"
```

Key Tables (SQL Sketch):

sql

```
-- Core
CREATE TABLE companies (id SERIAL PRIMARY KEY, name VARCHAR);
```

```
CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR, role_id INT,
company_id INT);
CREATE TABLE roles (id SERIAL PRIMARY KEY, name VARCHAR, company_id
INT);

-- Inventory
CREATE TABLE inventory_items (id SERIAL PRIMARY KEY, name VARCHAR, code
VARCHAR UNIQUE, price DECIMAL, company_id INT);
CREATE TABLE branches (id SERIAL PRIMARY KEY, name VARCHAR, company_id
INT);
CREATE TABLE stock (item_id INT, branch_id INT, quantity INT, PRIMARY
KEY(item_id, branch_id));

-- Sales
CREATE TABLE customers (id SERIAL PRIMARY KEY, name VARCHAR, company_id
INT);
CREATE TABLE sales_invoices (id SERIAL PRIMARY KEY, customer_id INT,
branch_id INT, total DECIMAL);
CREATE TABLE sales_items (invoice_id INT, item_id INT, qty INT, price
DECIMAL);
```

3.2 Prototype ERD (Simplified)

Only essentials:

text

```
erDiagram
    users ||--o{ inventory_items : "manages"
    users ||--o{ sales_invoices : "creates"
    inventory_items ||--o{ sales_items : "sold_in"
    sales_invoices ||--o{ sales_items : "contains"
    inventory_items ||--o{ stock : "has"
```

SQL Sketch (Copy-Paste Ready):

sql

```
CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR(100), role
VARCHAR(50));
```

```
CREATE TABLE inventory_items (id SERIAL PRIMARY KEY, name VARCHAR(100),
code VARCHAR(50) UNIQUE, price DECIMAL, stock_qty INT DEFAULT 0);
CREATE TABLE sales_invoices (id SERIAL PRIMARY KEY, customer_name
VARCHAR(100), total DECIMAL, created_at TIMESTAMP DEFAULT NOW());
CREATE TABLE sales_items (invoice_id INT, item_id INT, qty INT, price
DECIMAL);
```

---

4. Use Case Diagrams (Textual + Mermaid)

4.1 Full Project Use Cases

Actors: Admin, Inventory Manager, Sales User.edrawmax.wondershare+2

text

```
graph TD
    Admin --> Login
    Admin --> ManageUsers
    Admin --> ManageModules
    Admin --> ViewReports
    InventoryManager --> ManageInventory
    InventoryManager --> ViewStock
    SalesUser --> CreateSale
    SalesUser --> ManageCustomers
```

List:

1. **Login** (all).
2. **Manage Users/Roles** (Admin).
3. **Manage Modules** (Admin).
4. **Manage Inventory Items/Stock** (Inventory Manager).
5. **Create Sales Invoice** (Sales User).
6. **View Dashboard/Reports** (all).
7. **Manage Branches** (Admin).
8. **Manage Purchases/Imports** (Import Clerk).

4.2 Prototype Use Cases (Week 2 Focus)

text

```
graph TD
```

```
Admin --> Login
Admin --> AddUser
Admin --> AddInventoryItem
SalesUser --> CreateSale
Admin --> ViewDashboard
```

List:

1. **Login** (hard-coded).
2. **Add/Edit Users** (Admin).
3. **Add/Edit Inventory Items** (Admin).
4. **Create Sale Invoice** (deducts stock).
5. **View Dashboard** (stats).

---

5. Project Suggestions (Essential Improvements)

Category	Suggestion	Why? / When?
UI/UX	Mantine `DataTable` for lists, `Stepper` for forms.	Fast, professional ERP look. Week 1. sitepoint+1
Data	Add `serial_number` to inventory_items.	Scientific equipment needs tracking. Full SRS.
Security	Rust: `argon2` for passwords, JWT for sessions.	Production-ready. Week 2.
Reports	Basic PDF export (rust-print or frontend jsPDF).	Client demo wow-factor. Post-prototype.
Testing	Frontend: Vitest; Backend: cargo test.	University marks. Week 2.
Docs	README.md + API docs (utoipa).	Client handover.

Anti-Scope Creep Rule: Any new feature → update SRS + add to roadmap. No code without SRS approval.

---

6. AI Integration Ideas (Phase 2+)

AI fits perfectly for an **equipment importer ERP**. Start simple, scale later.everworker+2

AI Feature	Description	Tech / Effort
Demand Forecasting	Predict stock needs based on sales history.	Rust + simple ML (linfa) or OpenAI API. Medium.
Low Stock Alerts	Auto-alert when stock < threshold (smart thresholds).	Rule-based → ML. Low.
Invoice OCR	Scan supplier invoices → auto-create purchase.	Tauri + Tesseract/OpenAI Vision. Medium.
Anomaly Detection	Flag unusual sales (theft/fraud).	Basic stats → ML. Low.
Chat Assistant	“Show low stock items” natural language query.	Ollama (local LLM) + Rust. High (but cool demo).

How to Add:

- Week 3+: Integrate **Ollama** (local AI) via Rust FFI.
- Call OpenAI API for vision/forecasting (add API key in settings).
- University bonus: Demo “AI-powered stock prediction”.